

Module 10: Exceptions

Java splits exceptions into two families and makes the compiler police one of them. It is nearly alone in this, and the argument about whether that was a good idea has run for twenty-five years.

This module covers the mechanics, states the argument fairly, and shows the three failure modes that cost people real debugging time: a swallowed exception, a lost cause, and a `close()` that hides the real problem.

What You'll Learn

- Checked versus unchecked, and how to decide which to throw
- `try / catch / finally`, multi-catch, and catch ordering
- try-with-resources, reverse close order, and suppressed exceptions
- Why `return` inside `finally` is banned in every style guide
- Custom exceptions, and the constructor argument people forget

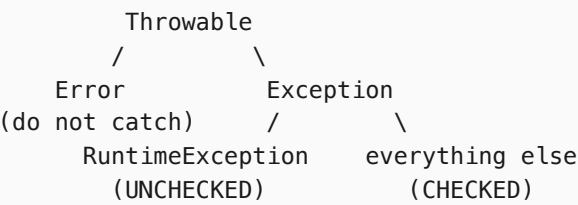
Coming From Another Language

	Python / JS	Java
Compiler-enforced handling	none	checked exceptions
Catch several types	<code>except (A, B)</code>	<code>catch (A B e)</code>
Cleanup	<code>finally</code> , <code>with</code>	<code>finally</code> , try-with-resources
Resource protocol	<code>__enter__ / __exit__</code>	<code>AutoCloseable</code>
Exception chaining	<code>raise X from e</code>	<code>new X(msg, cause)</code>

The one genuinely new idea is checked exceptions. Everything else you already know under a different name.

The Lesson

The two families



- **Unchecked** (`RuntimeException` and below): the compiler says nothing. These mean a programming mistake. The fix is to change the code, not to handle it.
- **Checked** (everything else under `Exception`): the compiler forces you to catch it or declare it.
- **Error**: the JVM is in trouble. `OutOfMemoryError`, `StackOverflowError`. Do not catch these, you cannot fix them from inside.

A detail worth knowing, from `CheckedVsUnchecked.java`:

```
unchecked: NumberFormatException -> For input string: "12x"
unchecked: NumberFormatException -> Cannot parse null string
```

`Integer.parseInt(null)` throws `NumberFormatException`, **not** `NullPointerException`. Guessing the type rather than reading the javadoc is how you write a `catch` block that never fires.

Multi-catch handles two types the same way, and the variable is implicitly final. **Catch order matters**: a supertype must come after its subtypes, and the compiler rejects an unreachable catch rather than letting it sit there.

The argument, fairly stated

For: a failure the caller can recover from should be visible in the type system. A file might not exist. That is not a bug, and the compiler making you think about it is a feature.

Against: in practice they push people towards `catch (Exception e) {}` to silence the compiler, which is strictly worse than no checking. They leak through abstractions, since adding a `throws` changes every caller's signature all the way up. And they compose badly with lambdas, which is why nothing in the streams API accepts a throwing function.

No language designed after Java has copied them. That is evidence, not proof.

What to do today:

- Unchecked for programming errors: `IllegalArgumentException`, `IllegalStateException`.
- Checked only when the caller can genuinely do something other than log and give up.
- Never declare `throws Exception`. It tells the caller nothing and forces them to catch everything.

try-with-resources

The happy path is unremarkable. The failure path is where the design shows. From

`TryWithResources.java`:

```
open A
open B
body running
close B (and failing)
close A (and failing)
caught: body failed
    suppressed: close B failed
    suppressed: close A failed
```

Two things to take from that:

1. **Resources close in reverse order.** B opened last, closed first. That matters when one resource wraps another.
2. **Close failures are suppressed, not substituted.** The body's exception is the one you catch; failures from `close()` attach to it and are readable with `getSuppressed()`.

Both exist because the hand-written equivalent got them wrong:

```
Resource r = null;
try {
    r = open();
    use(r);
} finally {
    if (r != null) r.close();    // if this throws, it REPLACES the real exception
}
```

A `close()` failure would hide the actual problem and the real cause was gone.

`close()` also runs when the body returns early, so you never need a `finally` for it.

finally, and how it eats exceptions

`finally` always runs. That is the promise and the trap. From [FinallySwallows.java](#):

```
static int swallowsTheException() {
    try {
        throw new RuntimeException("you will never see this");
    } finally {
        return 42;
    }
}
```

Output: `42`. The exception is gone. No log, no stack trace, nothing.

A `return` inside `finally` discards whatever the `try` block was doing, including an exception in flight. The same mechanism silently overwrites a good return value, which is much easier to miss in review.

Never put `return`, `break` or `continue` inside a `finally` block. Every style guide bans it and this is why.

Note also that `finally` does not run if the JVM exits. `System.exit(0)` inside a `try` skips it entirely. "Always runs" means within a normal method exit.

Custom exceptions and the cause

From `CustomExceptions.java`, the same wrapping done twice:

```
--- with the cause preserved ---
  caused by: java.io.IOException: users/7.json: no such file
  root cause: java.io.IOException: users/7.json: no such file

--- with the cause thrown away ---
  caused by: null
  the original IOException is gone. good luck.
```

The cause is the second constructor argument, and forgetting it is the most common mistake in exception handling. Without it, the stack trace starts where you rethrow and says nothing about what actually failed.

Always give a custom exception a constructor that takes a `Throwable` cause.

Carry structured data rather than encoding it in the message, so callers can ask for the id instead of parsing it back out of a string.

Checked or unchecked? `extends Exception` is checked, `extends RuntimeException` is not. Ask whether a caller can realistically do something other than log and give up.

Run It

```
java modules/10-exceptions/examples/CheckedVsUnchecked.java
java modules/10-exceptions/examples/TryWithResources.java
java modules/10-exceptions/examples/FinallySwallows.java
java modules/10-exceptions/examples/CustomExceptions.java

# Fails on purpose.
java modules/10-exceptions/examples/UnhandledChecked.java

./scripts/verify-examples.sh modules/10-exceptions
```

Common Mistakes

The empty catch block.

```
try { readConfig(path); } catch (IOException e) { }
```

It compiles, it silences the compiler, and it converts a visible failure into a program that quietly does the wrong thing. This is the worst line you can write in Java, and the fact that checked exceptions produce it is the strongest argument against the feature.

Losing the cause. `throw new MyException("failed")` inside a catch block discards everything you needed to debug it.

`return` in a `finally` block. Covered above. It eats exceptions.

Catching `Exception` to cover everything. You have just caught bugs alongside expected failures, including ones you have no idea how to handle.

Catching `Throwable` or `Error`. You cannot recover from `OutOfMemoryError`, and catching it turns a crash into a hang.

Using exceptions for control flow. Throwing to exit a loop is slow, because building a stack trace is expensive, and it hides the logic from the reader.

Declaring `throws Exception`. It tells callers nothing except that everything is now their problem.

Key Takeaways

- **Unchecked means a bug, checked means a foreseeable failure.** Choose by asking what the caller could actually do.
- **Never write an empty catch block.** If you truly cannot handle it, wrap it in an unchecked exception and let it travel.
- **`try-with-resources` closes in reverse order and suppresses rather than replaces,** which hand-written cleanup got wrong for years.
- `return` inside `finally` silently discards exceptions.
- **Always pass the cause** when wrapping, and always give custom exceptions a cause constructor.
- **Never declare `throws Exception`.**

Homework

[homework/README.md](#)

Build a small config loader, then reproduce three exception anti-patterns and fix each one. Reference solution in [solutions/10-exceptions/](#).